

MANIPULATOR Z ELEKROMAGNESEM

Raport końcowy

Grupa 1:

Zuzanna Głowacz 279574

Filip Żórawski 279586

Konrad Teplicki 274657

Norbert Pala 279610

Spis treści

1. ZAŁOŻENIA PROJEKTOWE	3
2. OBLICZENIA.....	4
3. CZĘŚĆ PROGRAMISTYCZNA	5
3.1 SERWER APLIKACYJNY I ZARZĄDZENIE STANEM (BACKEND WE FLASKU).....	5
3.1.1 Kod znajdujący się na serwerze.....	6
3.2 OPROGRAMOWANIE UKŁADOWE MIKROKONTROLERA (FIRMWARE ESP32)	10
3.2.1 Kod mikrokontrolera ESP32	11
4. CZĘŚĆ ELEKTRYCZNA.....	15
4.1 SCHEMAT BLOKOWY UKŁADU ELEKTRONICZNEGO.....	15
4.2 WYKAZ CZĘŚCI	15
4.3 SCHEMAT ELEKTRYCZNY I ZABEZPIECZENIA.....	16
4.4 WYKAZ WYPROWADZEŃ	18
5. KONSTRUKCJA MECHANICZNA MANIPULATORA	19
5.1 ZAŁOŻENIA PROJEKTOWE KONSTRUKCJI MECHANICZNEJ	19
5.2 MODEL 3D MANIPULATORA	19
5.3 OPIS ELEMENTÓW KONSTRUKCYJNYCH	21
5.3.1 Podstawa	21
5.3.2 Pierwszy człon manipulatora	22
5.3.3 Drugi człon manipulatora	24
5.3.4 Trzeci człon manipulatora z miejscem na elektromagnes	25
5.4 ELEMENTY ZŁĄCZNE.....	26
5.5 PROCES MONTAŻU MANIPULATORA	26
5.6 GOTOWY ZMONTOWANY MANIPULATOR	27
6. WNIOSKI	28

1. Założenia projektowe

Projekt miał na celu wykorzystanie wiedzy zdobytej w różnych dziedzinach mechatroniki do stworzenia robota zawierającego w sobie części z trzech głównych dziedzin; elektroniki, programowania oraz mechaniki. Projektem realizowanym było ramie robota z elektromagnesem umieszczonym jako efektor. Robot dzięki wykorzystaniu bazy danych umieszczonej na serwerze mógł wykonywać ruchy członami oraz mógł on łapać metalowe przedmioty (np. śrubki). Jego zastosowaniem mogłoby być sortowanie małych elementów metalowych lub ich składowanie.

2. Obliczenia

Obliczenie momentów obciążających przeguby manipulatora

Dane wejściowe:

- długość członu 1: $L_1=100 \text{ mm} = 10 \text{ cm}$
- długość członu 2: $L_2=150 \text{ mm} = 15 \text{ cm}$
- długość członu 3: $L_3=150 \text{ mm} = 15 \text{ cm}$
- masa serwa: $m_s=55,2 \text{ g} = 0,0552 \text{ kg}$
- masa elektromagnesu: $m_e=25 \text{ g} = 0,025 \text{ kg}$

Przyjęto orientacyjne masy drukowanych członów:

- człon 1: $m_1=20 \text{ g} = 0,020 \text{ kg}$
- człon 2: $m_2=25 \text{ g}=0,025 \text{ kg}$
- człon 3: $m_3=25 \text{ g}=0,025 \text{ kg}$

Moment obrotowy obliczono ze wzoru:

$$M=m \cdot r$$

gdzie:

- M — moment obrotowy [$\text{kg} \cdot \text{cm}$],
- m — masa elementu [kg],
- r — odległość środka masy od osi przegubu [cm].

Przyjęto najniekorzystniejszy przypadek pracy manipulatora, w którym wszystkie człony są ustawione poziomo. W takim położeniu momenty od siły ciężkości są największe.

Moment w przegubie 3

Przegub 3 obciążony jest masą elektromagnesu oraz masą członu 3. Środek masy członu 3 znajduje się w połowie jego długości.

$$M_3 = m_e \cdot L_3 + m_3 \cdot \frac{L_3}{2}$$

$$M_3 = 0,025 \cdot 15 + 0,025 \cdot 7,5$$

$$M_3 = 0,375 + 0,1875$$

$$M_3 = 0,5625 \text{ kg} \cdot \text{cm}$$

Moment w przegubie 2

Przegub 2 obciążony jest członem 2, serwem przegubu 3, członem 3 oraz elektromagnesem.

$$M2 = m2 \cdot L2/2 + ms \cdot L2 + m3 \cdot (L2 + L3/2) + me \cdot (L2 + L3)$$

$$M2 = 0,025 \cdot 7,5 + 0,0552 \cdot 15 + 0,025 \cdot 22,5 + 0,025 \cdot 30$$

$$M2 = 0,1875 + 0,828 + 0,5625 + 0,75$$

$$M2 = 2,328kg \cdot cm$$

Moment w przegubie 1

Przegub 1 obciążony jest członem 1, serwem przegubu 2, członem 2, serwem przegubu 3, członem 3 oraz elektromagnesem.

$$M1 = \frac{m1 \cdot L1}{2} + ms \cdot L1 + m2 \cdot \left(L1 + \frac{L2}{2}\right) + ms \cdot (L1 + L2) + m3 \cdot \left(L1 + L2 + \frac{L3}{2}\right) + me \cdot (L1 + L2 + L3)$$

$$M1 = 0,1 + 0,552 + 0,4375 + 1,38 + 0,8125 + 1,0$$

$$M1 = 4,282kg \cdot cm$$

Oznaczenie przegubu	Moment w przegubie
Przegub 1	4,28 kg·cm
Przegub 2	2,33 kg·cm
Przegub 3	0,56 kg·cm

Zastosowane serwa mają moment:

Ms=15 kg·cm przy 6V

Największe obciążenie występuje w przegubie 1:

$$(4,28kg \cdot cm / 15kg \cdot cm) \cdot 100\% = 28,5\%$$

Oznacza to, że w najniekorzystniejszym położeniu manipulator wykorzystuje około:

28,5% nominalnego momentu serwa.

W przypadku chęci obciążenia manipulatora ładunkiem w postaci śrubek, których masa nie przekracza 100g = 0,1kg (nazywana dalej masą - m_p), momenty w poszczególnych członach będą posiadały dodatkowy składnik w postaci:

$$\Delta M_3 = m_p \cdot 15$$

$$\Delta M_2 = m_p \cdot 30$$

$$\Delta M_1 = m_p \cdot 40$$

W takim przypadku moment maksymalny, który występuje w przegubie numer jeden wzrośnie do wartości 8,28kg*cm. Oznacza to, że serwo będzie wykorzystywać około 55% swojego nominalnego momentu.

Na podstawie przeprowadzonych obliczeń można stwierdzić, że zastosowane serwa o momencie 15 kg·cm przy zasilaniu 6 V są wystarczające do napędu analizowanego manipulatora. Największe obciążenie występuje w pierwszym przegubie, ponieważ przenosi on moment pochodzący od wszystkich dalszych członów, serw oraz elektromagnesu. Nawet przy dodatkowym obciążeniu końcówki roboczej masą 100 g moment w pierwszym przegubie wynosi około 8,28 kg·cm, co stanowi około 55% nominalnego momentu serwa.

3. Część programistyczna

Oprogramowanie systemu sterowania manipulatorem zostało podzielone na dwa niezależne środowiska uruchomieniowe komunikujące się ze sobą za pomocą bezstanowego protokołu HTTP. Taka separacja (architektura klient-serwer) pozwala na całkowite uniezależnienie interfejsu użytkownika od fizycznej lokalizacji robota.

3.1 Serwer aplikacyjny i zarządzanie stanem (Backend we Flasku)

Sercem systemu odpowiedzialnym za przyjmowanie rozkazów od operatora oraz ich dystrybucję do robota jest aplikacja sieciowa napisana w języku Python z wykorzystaniem mikro-frameworka Flask, uruchomiona w chmurze na platformie hostingowej PythonAnywhere.

Ze względu na restrykcje środowiska darmowego w celach optymalizacyjnych zaimplementowano hybrydowe podejście do przechowywania danych strukturalnych. Zamiast relacyjnej bazy danych na dysku, aktualna matryca stanów manipulatora została zamapowana bezpośrednio w pamięci podręcznej RAM serwera w postaci globalnego słownika Pythona (`stan_robota`).

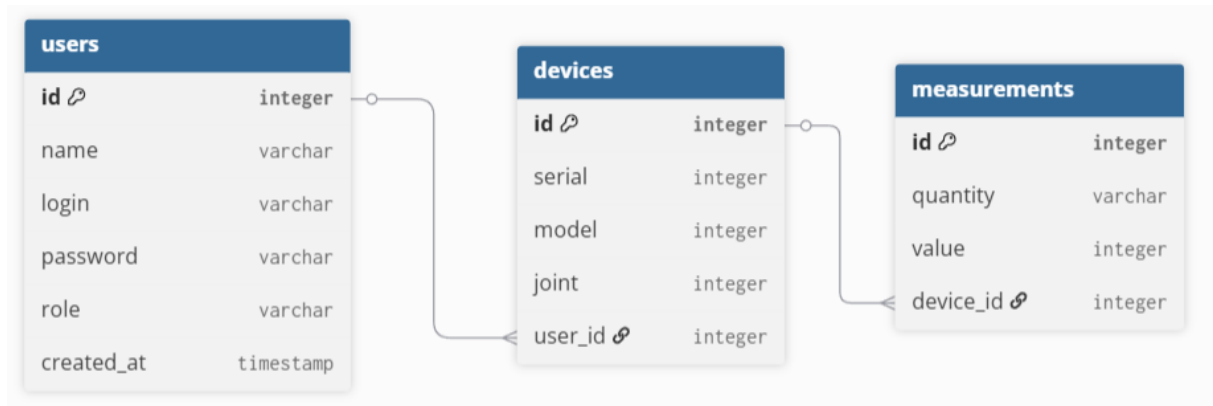
Endpointy API (REST):

Serwer wystawia interfejs programistyczny składający się z dedykowanych tras. Trasa `POST /zapisz_rozkaz_panelu` odbiera asynchroniczne zapytania w formacie JSON generowane przez suwaki JavaScript z poziomu przeglądarki i aktualizuje stany poszczególnych pinów. Z kolei trasa `GET /device/1/joint/<int:joint_index>/latest` pozwala na bezautoryzacyjny, szybki odczyt najnowszych nastaw parametrów przez mikrokontroler.

Mechanizm autoryzacji:

Dostęp do graficznego interfejsu użytkownika (`GET /`) został zabezpieczony kryptograficznie przy użyciu mechanizmu sesji Flaska (`flask.session`) z kluczem cyfrowym. Dane użytkowników (admin) sprawdzane są w czasie rzeczywistym bezpośrednio w kodzie backendu, co uniemożliwia przejęcie kontroli nad manipulatorem przez osoby niepowołane.

Database:



3.1.1 Kod znajdujący się na serwerze

```
from flask import Flask, redirect, render_template, request, jsonify, session, url_for, render_template_string
from datetime import datetime

app = Flask(__name__)
# Klucz do szyfrowania sesji logowania
app.secret_key = 'super-tajny-klucz-bez-bazy-danych'

# --- AKUALNY STAN ROBOTA (Zamiast bazy danych, trzymamy w pamięci RAM) ---
# Domyślnie serwa (25, 26, 27) na 90 stopni, magnes (99) na 0.0 (wyłączony)
stan_robota = {
    27: {"value": 90.0, "quantity": "angle", "unit": "deg"},
    26: {"value": 90.0, "quantity": "speed", "unit": "raw"},
    25: {"value": 90.0, "quantity": "speed", "unit": "raw"},
    99: {"value": 0.0, "quantity": "state", "unit": "bin"}
}

# --- SŁOWNIK Z UŻYTKOWNIKAMI (Na sztywno w kodzie) ---
UZYTKOWNICY = {
    "admin": "admin",
    "Orthoget": "alamakota"
}

# =====
# 1. TRASY LOGOWANIA I STRONY WWW
# =====

@app.route("/login", methods=["GET", "POST"])
def login():
    error = None
    if request.method == "POST":
        input_username = request.form["username"]
        input_password = request.form["password"]

    # Weryfikacja "na sztywno" na podstawie naszego słownika
```



```

if input_username in UZYTKOWNICY and UZYTKOWNICY[input_username] == input_password:
    session["zalogowany"] = True
    session["username"] = input_username
    return redirect(url_for("panel_sterowania"))
else:
    error = "Niepoprawny login lub hasło! Spróbuj ponownie."

html_login = """
<!DOCTYPE html>
<html lang="pl">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Logowanie - Robot IoT</title>
    <style>
        body { font-family: 'Segoe UI', Arial, sans-serif; background: #1e1e24; color: #fff; display: flex; justify-
content: center; align-items: center; height: 100vh; margin: 0; }
        .login-box { background: #2a2a35; padding: 40px; border-radius: 15px; box-shadow: 0 8px 24px
rgba(0,0,0,0.4); width: 320px; text-align: center; }
        h2 { color: #00adb5; margin-bottom: 25px; }
        input[type="text"], input[type="password"] { width: 100%; padding: 12px; margin: 10px 0; border-radius:
8px; border: 1px solid #444; background: #222; color: #fff; box-sizing: border-box; }
        .btn-login { width: 100%; padding: 12px; background: #00adb5; border: none; color: white; font-weight:
bold; border-radius: 8px; cursor: pointer; font-size: 16px; margin-top: 15px; transition: 0.2s; }
        .btn-login:hover { background: #008c95; }
        .error-msg { color: #d9534f; margin-bottom: 15px; font-size: 14px; font-weight: bold; }
    </style>
</head>
<body>
    <div class="login-box">
        <h2><img alt="Robot icon" data-bbox="208 568 225 585"/> Autoryzacja</h2>
        {% if error %}
            <div class="error-msg">{{ error }}</div>
        {% endif %}
        <form method="post">
            <input type="text" name="username" placeholder="Nazwa użytkownika (Login)" required autofocus>
            <input type="password" name="password" placeholder="Hasło" required>
            <button type="submit" class="btn-login">Zaloguj się</button>
        </form>
    </div>
</body>
</html>
"""

return render_template_string(html_login, error=error)

@app.route("/logout")
def logout():
    session.pop("zalogowany", None)
    session.pop("username", None)
    return redirect(url_for("login"))

```

```

@app.route("/", methods=["GET"])
def panel_sterowania():
    if not session.get("zalogowany"):
        return redirect(url_for("login"))

    html_panel = """
<!DOCTYPE html>
<html lang="pl">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Panel Sterowania Robotem IPZ</title>
    <style>
        body { font-family: 'Segoe UI', Arial, sans-serif; background: #1e1e24; color: #fff; text-align: center;
padding: 20px; }
        .container { max-width: 600px; margin: 0 auto; background: #2a2a35; padding: 30px; border-radius: 15px;
box-shadow: 0 8px 24px rgba(0,0,0,0.3); }
        .header-flex { display: flex; justify-content: space-between; align-items: center; margin-bottom: 30px;
border-bottom: 2px solid #333; padding-bottom: 15px; }
        h1 { color: #00adb5; margin: 0; font-size: 24px; }
        .logout-link { color: #d9534f; text-decoration: none; font-weight: bold; border: 1px solid #d9534f; padding:
5px 12px; border-radius: 5px; transition: 0.2s; }
        .logout-link:hover { background: #d9534f; color: white; }
        .control-card { background: #333344; padding: 20px; margin-bottom: 20px; border-radius: 10px; text-align:
left; }
        .control-card label { font-size: 18px; font-weight: bold; display: block; margin-bottom: 10px; }
        .slider-container { display: flex; align-items: center; gap: 15px; }
        input[type=range] { flex-grow: 1; height: 8px; border-radius: 5px; accent-color: #00adb5; cursor: pointer; }
        .val-display { font-size: 20px; font-weight: bold; min-width: 50px; text-align: right; color: #00adb5; }
        .btn { display: block; width: 100%; padding: 15px; font-size: 20px; font-weight: bold; color: white; border:
none; border-radius: 8px; cursor: pointer; transition: 0.2s; }
        .btn-on { background: #393e46; border: 2px solid #00adb5; color: #00adb5; }
        .btn-on:hover { background: #00adb5; color: white; }
        .btn-danger { background: #d9534f; }
        .btn-danger:hover { background: #c9302c; }
        .user-info { font-size: 14px; color: #888; }
    </style>
</head>
<body>
    <div class="container">
        <div class="header-flex">
            <div>
                <h1> Panel Sterowania Robotem </h1>
                <span class="user-info">Zalogowany jako: <b>{{ session['username'] }}</b></span>
            </div>
            <a href="/logout" class="logout-link">Wyloguj</a>
        </div>

        <div class="control-card">

```

```

<label><img alt="Servo motor icon" data-bbox="238 88 255 100"/> Serwo A (Podstawa - Pin 27)</label>
<div class="slider-container">
  <input type="range" min="0" max="180" value="90" id="servoA" onchange="wyslijRozkaz(27,
this.value, 'angle', 'deg')">
  <span class="val-display" id="valA">90°</span>
</div>
</div>

<div class="control-card">
  <label><img alt="Servo motor icon" data-bbox="238 232 255 244"/> Serwo B (Ramie - Pin 26)</label>
  <div class="slider-container">
    <input type="range" min="0" max="180" value="90" id="servoB" onchange="wyslijRozkaz(26,
this.value, 'speed', 'raw')">
    <span class="val-display" id="valB">90</span>
  </div>
</div>

<div class="control-card">
  <label><img alt="Servo motor icon" data-bbox="238 376 255 388"/> Serwo C (Efektor - Pin 25)</label>
  <div class="slider-container">
    <input type="range" min="0" max="180" value="90" id="servoC" onchange="wyslijRozkaz(25,
this.value, 'speed', 'raw')">
    <span class="val-display" id="valC">90</span>
  </div>
</div>

<div class="control-card" style="text-align:center;">
  <label style="text-align:left;"><img alt="Electromagnet icon" data-bbox="382 520 399 532"/> Elektromagnes (Pin 32)</label>
  <div style="display:flex; gap:10px; margin-top:10px;">
    <button class="btn btn-on" onclick="wyslijRozkaz(99, 1.0, 'state', 'bin')">ZłAP (Wł)</button>
    <button class="btn btn-danger" onclick="wyslijRozkaz(99, 0.0, 'state', 'bin')">PUŚĆ (WYł)</button>
  </div>
</div>
</div>

<script>
  document.getElementById('servoA').oninput = function() { document.getElementById('valA').innerText =
this.value + '°'; }
  document.getElementById('servoB').oninput = function() { document.getElementById('valB').innerText =
this.value; }
  document.getElementById('servoC').oninput = function() { document.getElementById('valC').innerText =
this.value; }

  function wyslijRozkaz(pinIndex, wartosc, cecha, jednostka) {
    fetch('/zapisz_rozkaz_panelu', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        pin: pinIndex,
        value: parseFloat(wartosc),
        quantity: cecha,

```

```

        unit: jednostka
    })
})
.then(res => res.json())
.then(data => console.log("Zapisano w RAM:", data))
.catch(err => console.error("Błąd zapisu:", err));
    }
</script>
</body>
</html>
"""

return render_template_string(html_panel)

@app.route("/zapisz_rozkaz_panelu", methods=["POST"])
def zapisz_rozkaz_panelu():
    if not session.get("zalogowany"):
        return jsonify({"status": "error", "message": "Brak autoryzacji"}), 401

    pin = int(request.json['pin'])
    value = request.json['value']
    quantity = request.json['quantity']
    unit = request.json['unit']

    # Aktualizujemy globalny słownik w pamięci serwera
    stan_robota[pin] = {
        "value": value,
        "quantity": quantity,
        "unit": unit
    }

    return jsonify({"status": "success", "saved_value": value, "pin": pin})

# =====
# 2. ENDPOINT DLA ROBOTA ESP32 (Bez logowania)
# =====

@app.route("/device/<int:device_id>/joint/<int:joint_index>/latest", methods=["GET"])
def get_latest_joint_command(device_id, joint_index):
    # Zwracamy dane prosto z naszego słownika w RAM
    if joint_index in stan_robota:
        return jsonify(stan_robota[joint_index]), 200
    else:
        # Awaryjna domyślna wartość, jeśli ESP32 zapyta o dziwny pin
        return jsonify({"value": 90.0 if joint_index != 99 else 0.0}), 200

```

3.2 Oprogramowanie układowe mikrokontrolera (Firmware ESP32)

Program sterujący układem wbudowanym został zaimplementowany w języku C++ w środowisku Arduino IDE i wgrany na moduł ESP32, który pełni rolę fizycznego sterownika peryferiów.

Głównym zadaniem mikrokontrolera jest praca w pętli cyklicznego odpytywania serwera (tzw. polling). Co około 800 milisekund ESP32 inicjuje połączenie klienckie przez wbudowany stos Wi-Fi, wysyłając zapytania HTTP GET do chmury.

Przetwarzanie danych (Parsowanie JSON):

Po odebraniu tekstowej odpowiedzi z serwera, firmware wykorzystuje wydajną bibliotekę ArduinoJson do deserializacji strumienia danych. Z obiektu JSON wyciągana jest zmienna zmiennoprzecinkowa "value", reprezentująca docelowy kąt lub stan logiczny danego elementu.

Algorytm płynnego sterowania prędkością:

Standardowe, bezpośrednie przekazanie współrzędnych do serwomechanizmu za pomocą komendy write() skutkuje gwałtownym skokiem prądu i szarpnięciami konstrukcji. Aby tego uniknąć, w kodzie zaimplementowano autorski algorytm interpolacji krokowej. Mikrokontroler porównuje pozycję aktualną z pozycją docelową pobraną z sieci i za pomocą pętli warunkowych modyfikuje sygnał PWM o $\pm 1^\circ$. Wprowadzenie mikrosekundowych opóźnień (delay) pomiędzy krokami pozwoliło na płynną regulację prędkości kątowej ramion, co zniwelowało drgania mechaniczne i zabezpieczyło porty GPIO przed przeciążeniem.

Sterowanie dyskretne:

Poza generowaniem sygnałów PWM dla trzech serwomechanizmów (piny 25, 26, 27), program realizuje dwustanowe sterowanie cyfrowe (HIGH/LOW) na pinie 32, które poprzez zewnętrzny układ kluczujący (tranzystor/przekaźnik) załącza pole magnetyczne efektora końcowego (elektromagnesu).

3.2.1 Kod mikrokontrolera ESP32

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>
#include <ESP32Servo.h>

// --- KONFIGURACJA WI-FI ---
const char* ssid = "AndroidAP2934";
const char* haslo = "NeroiGucio";

// --- USTALENIE PINÓW ---
const int PIN_SERVO_A = 27;
```

```

const int PIN_SERWO_B = 26;
const int PIN_SERWO_C = 25;
const int PIN_MAGNES = 32;

// Obiekty dla trzech serwomechanizmów
Servo serwoA;
Servo serwoB;
Servo serwoC;

// Pomocnicza funkcja do pobierania wartości z bazy danych dla konkretnego pinu
float pobierzWartoscZSerwera(int pinIndex, float wartoscDomyslna) {
    HTTPClient http;
    // Dynamicznie podstawiamy numer pinu (joint_index) do adresu URL
    String url = "http://ipzelektromagnes.pythonanywhere.com/device/1/joint/" + String(pinIndex) + "/latest";
    http.begin(url);

    int httpCode = http.GET();
    float wyjściowaWartosc = wartoscDomyslna;

    if (httpCode == 200) {
        String payload = http.getString();
        JsonDocument doc;
        DeserializationError error = deserializeJson(doc, payload);

        if (!error) {
            wyjściowaWartosc = doc["value"];
        }
    } else {
        Serial.print("Błąd pobierania dla pinu ");
        Serial.print(pinIndex);
        Serial.print(" - Status HTTP: ");
        Serial.println(httpCode);
    }

    http.end();
    return wyjściowaWartosc;
}

void setup() {
    Serial.begin(115200);
    delay(1000);

    // 1. Inicjalizacja trzech serwomechanizmów
    serwoA.attach(PIN_SERWO_A, 500, 2400);
    serwoB.attach(PIN_SERWO_B, 500, 2400);
    serwoC.attach(PIN_SERWO_C, 500, 2400);

    // Ustawienie pozycji startowych (90 stopni = bezpieczny środek / stop)
    serwoA.write(90);
    serwoB.write(90);
    serwoC.write(90);
}

```

```

// 2. Inicjalizacja pinu elektromagnesu jako WYJŚCIE
pinMode(PIN_MAGNES, OUTPUT);
digitalWrite(PIN_MAGNES, LOW); // Na starcie magnes wyłączony

// 3. Łączenie z Wi-Fi
WiFi.begin(ssid, haslo);
Serial.print("Łączenie z Wi-Fi");
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println("\nPołączono z Wi-Fi! Robot gotowy do pracy.");
}

void loop() {
    if (WiFi.status() == WL_CONNECTED) {

        // --- 1. OBSŁUGA SERWA A (Pin 27) ---
        float katA = pobierzWartoscZSerwera(27, 90.0);
        if (katA >= 0 && katA <= 180) {
            serwoA.write((int)katA);
        }

        // --- 2. OBSŁUGA SERWA B (Pin 26) ---
        float katB = pobierzWartoscZSerwera(26, 90.0);
        if (katB >= 0 && katB <= 180) {
            serwoB.write((int)katB);
        }

        // --- 3. OBSŁUGA SERWA C (Pin 25) ---
        float katC = pobierzWartoscZSerwera(25, 90.0);
        if (katC >= 0 && katC <= 180) {
            serwoC.write((int)katC);
        }

        // --- 4. OBSŁUGA ELEKTROMAGNESU (Pin 32 / w panelu oznaczony jako 99) ---
        float stanMagnesu = pobierzWartoscZSerwera(99, 0.0);
        if (stanMagnesu == 1.0) {
            digitalWrite(PIN_MAGNES, HIGH); // Włącz przekaźnik / tranzystor magnesu
            Serial.println("Magnes: WŁĄCZONY");
        } else {
            digitalWrite(PIN_MAGNES, LOW); // Wyłącz
            Serial.println("Magnes: WYŁĄCZONY");
        }

        // Podgląd diagnostyczny w Serial Monitorze
        Serial.printf("Pozycje -> A: %.0f° | B: %.0f | C: %.0f\n", katA, katB, katC);
        Serial.println("-----");
    }
}

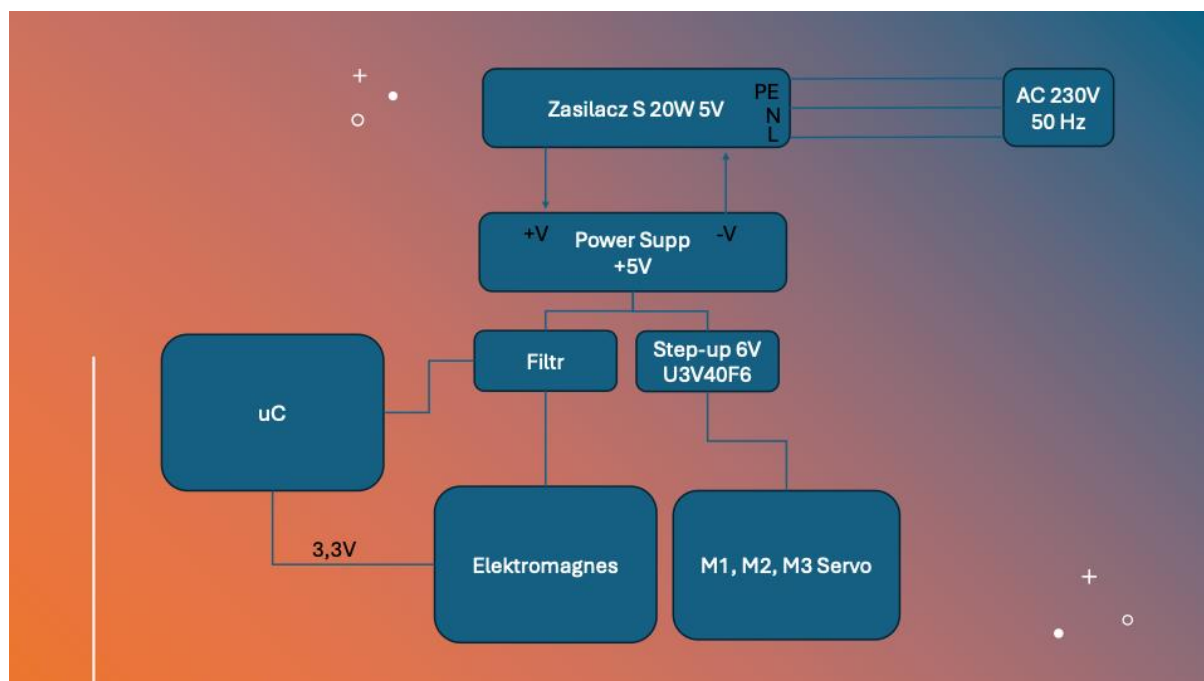
```

```
delay(800); // Odpytuj bazę danych o cały komplet poleceń co niecałą sekundę  
}
```


4. Część elektryczna

Projekt układu elektronicznego manipulatora musiał uwzględniać specyfikę pracy urządzeń indukcyjnych i wykonawczych (silniki, cewki), które charakteryzują się dużym, impulsowym poborem prądu oraz generowaniem zakłóceń elektromagnetycznych. Głównym celem było zapewnienie stabilnych warunków pracy dla wrażliwego mikrokontrolera ESP32.

4.1 Schemat blokowy układu elektronicznego



Rysunek 1 - Schemat blokowy reprezentujący połączenia i zależności między komponentami

Ogólna zasada działania:

Obwód logiki (5V): Zasilany bezpośrednio z wejścia, obsługuje ESP32 oraz diodę kontrolną.

Obwód mocy (6V): Zasilany przez przetwornicę Step-Up Pololu, dedykowany dla serwomechanizmów 2x MG-995 i AR-3606HB, co zapewnia im maksymalny moment obrotowy i izoluje logikę od zakłóceń silników.

4.2 Wykaz części

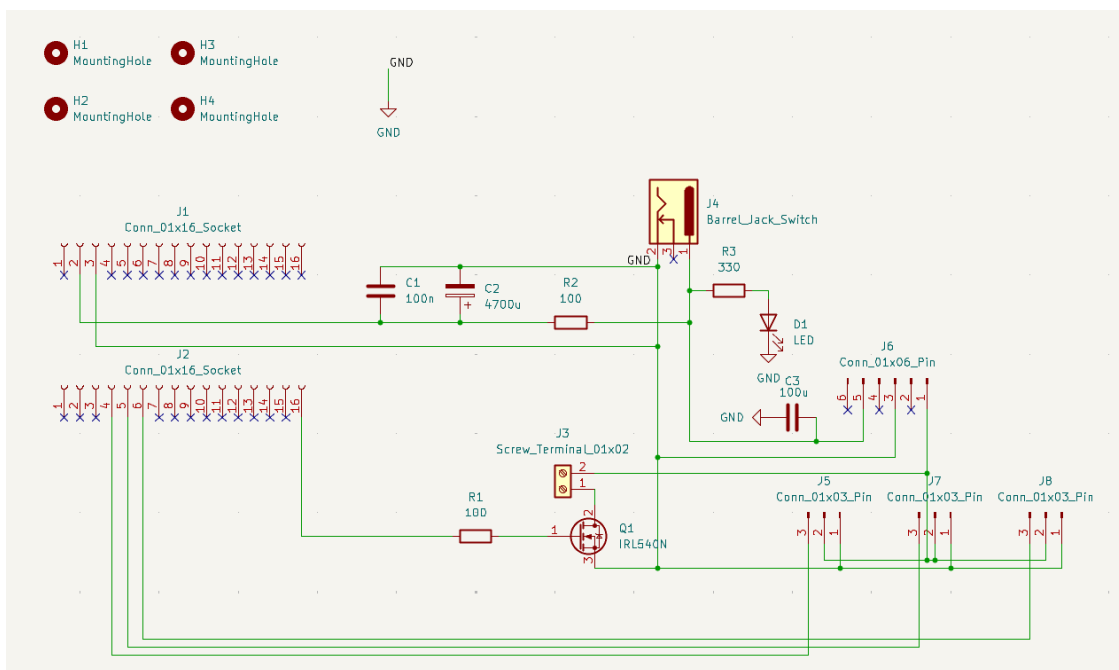
Elementy wykorzystywane:

1. Zasilanie
 - a. 1x Zasilacz S 20W 5 - 5V/4A AC/DC do układu uC
 - b. 1x Przetwornica step-up 6V 4A do układu
2. Sterowanie
 - a. 1x ESP32 do sterowania PWM serwonapędów oraz sterowania przełączaniem działania elektromagnesu

3. Aktuatory
 - a. 2x Serwo MG-995 6V
 - b. 1x Serwo AR-3606HB 6V
 - c. 1x Elektromagnes 6V
4. Komponenty PLB
 - a. 1x Tranzystor N-MOSFET, otwiera się przy 3.3V
 - b. 1x Kondensator elektrolityczny 4700 μ F
 - c. 1x Kondensator ceramiczny 100 nF
 - d. 1x Kondensator ceramiczny 100 μ F
 - e. 2x Rezystor 100
 - f. 1x Rezystor 330k
5. Złącza
 - a. 1x Złącze śrubowe (Screw Terminal) 2-pin, do połączenia elektromagnesu
 - b. 3x Złącze śrubowe (Screw Terminal) 3-pin, dla serw M1, M2, M3
 - c. 2x Listwy Goldpin żeńskie (1x16 pin), do wpięcia ESP i ochrony
 - d. 1x Barrel Jack złącze, do zasilania
 - e. 1x Listwa Goldpin żeńska(1x6), do połączenia przetwornicy

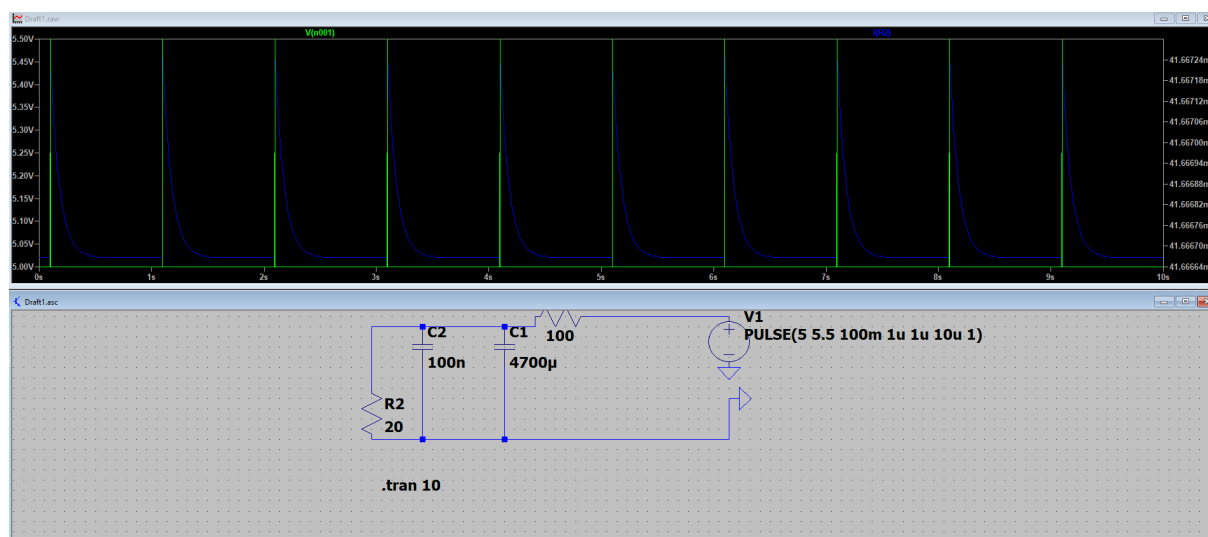
4.3 Schemat elektryczny i zabezpieczenia

Do stworzenia układu elektrycznego skorzystano z programu KiCAD, w którym stworzono i zamodelowano płytkę do późniejszego jej trawienia.



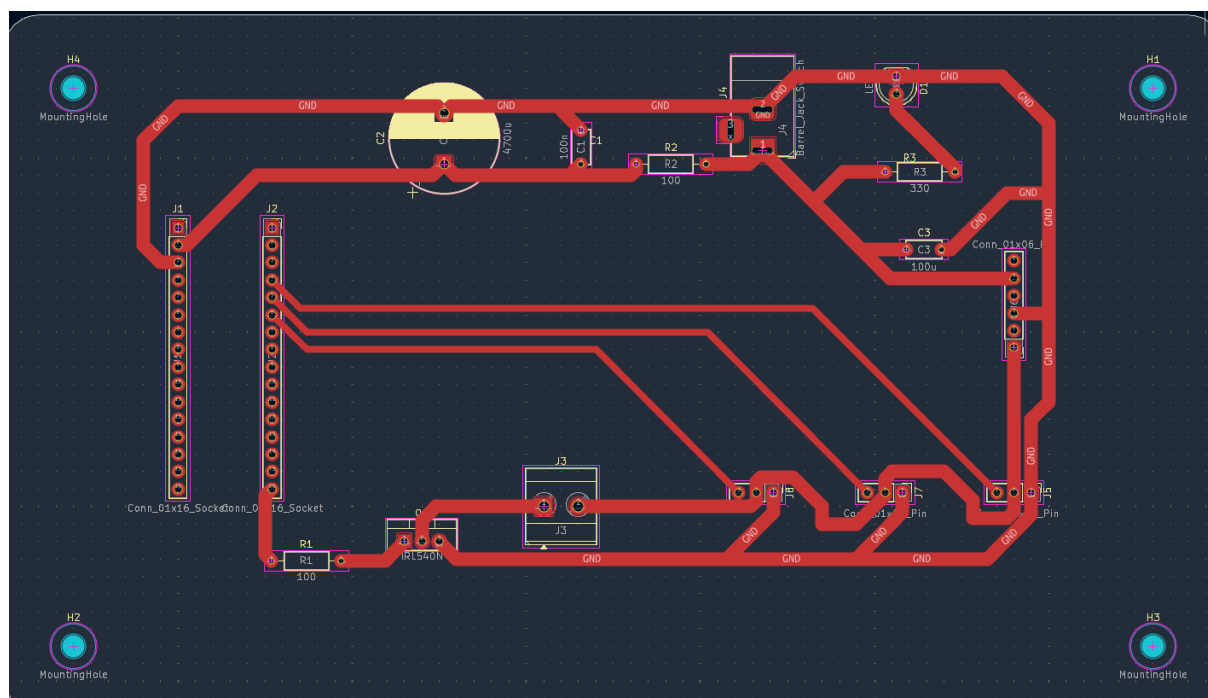
Rysunek 2 - Schemat elektryczny w programie KiCAD

Zabezpieczenie układu:



Rysunek 3 - Symulacja w programie LtSpice realizująca tłumienie zakłóceń

Schemat PCB:



Rysunek 4 - Wizualizacja płytki PCB

4.4 Wykaz wyprowadzeń

Sygnały sterujące i wykonawcze zostały przypisane do konkretnych portów GPIO mikrokontrolera ESP32 zgodnie z poniższą specyfikacją:

Komponent wykonawczy	Pin GPIO (ESP32)	Typ sygnału	Rola w układzie
Serwo A	GPIO 27	Wyjście PWM (50 Hz)	Obrót podstawy / ramienia głównego
Serwo B	GPIO 26	Wyjście PWM (50 Hz)	Ruch członu pośredniego
Serwo C	GPIO 25	Wyjście PWM (50 Hz)	Ruch członu krańcowego
Elektromagnes	GPIO 32	Wyjście cyfrowe (Dyskretne)	Załączanie pola chwytaka poprzez klucz tranzystorowy

5. Konstrukcja mechaniczna manipulatora

W niniejszym rozdziale przedstawiono projekt mechaniczny manipulatora, obejmujący model 3D, opis wszystkich elementów konstrukcyjnych oraz sposób ich montażu. Celem jest umożliwienie odtworzenia konstrukcji przez inną osobę wyłącznie na podstawie dokumentacji.

5.1 Założenia projektowe konstrukcji mechanicznej

Podczas projektowania części mechanicznej manipulatora przyjęto następujące założenia:

- zaprojektowanie manipulatora o trzech stopniach swobody umożliwiającego chwytanie i przenoszenie niewielkich elementów za pomocą elektromagnesu,
- zapewnienie udźwigu co najmniej 150 g, zgodnie z założeniami projektowymi całego stanowiska,
- konstrukcji do zastosowanych serwomechanizmów MG995 oraz AR-3606HB,
- wykonanie wszystkich elementów metodą druku 3D z filamentu PET-G, zapewniającego odpowiednią wytrzymałość mechaniczną oraz łatwość wykonania,
- zaprojektowanie elementów w sposób umożliwiający łatwy montaż i demontaż,
- ograniczenie masy ruchomych elementów w celu zmniejszenia obciążenia serwomechanizmów,
- zapewnienie odpowiedniej sztywności konstrukcji przy zachowaniu niewielkiej masy,
- umożliwienie prowadzenia przewodów wewnątrz lub wzdłuż konstrukcji w sposób nieograniczający ruchów manipulatora,
- przygotowanie projektu tak, aby wszystkie elementy mogły zostać wykonane z wykorzystaniem ogólnodostępnej drukarki 3D oraz standardowych elementów złącznych.

5.2 Model 3D manipulatora

Model manipulatora został wykonany w programie CAD. Konstrukcja składa się z podstawy, ramion zawierających mocowania serwomechanizmów oraz mocowanie elektromagnesu. Wszystkie elementy zostały zaprojektowane tak, aby mogły zostać wykonane metodą druku 3D oraz łatwo zmontowane przy użyciu standardowych śrub.



Rysunek 5. Model złożenia.



Rysunek 6. Model złożenia.

5.3 Opis elementów konstrukcyjnych

W niniejszym rozdziale przedstawiono wszystkie elementy składające się na konstrukcję mechaniczną manipulatora. Dla każdej części zaprezentowano model 3D oraz opis jej funkcji, przeznaczenia i sposobu współpracy z pozostałymi elementami. Konstrukcja została zaprojektowana z myślą o łatwym wykonaniu metodą druku 3D, prostym montażu oraz zapewnieniu odpowiedniej sztywności i wytrzymałości podczas pracy manipulatora.

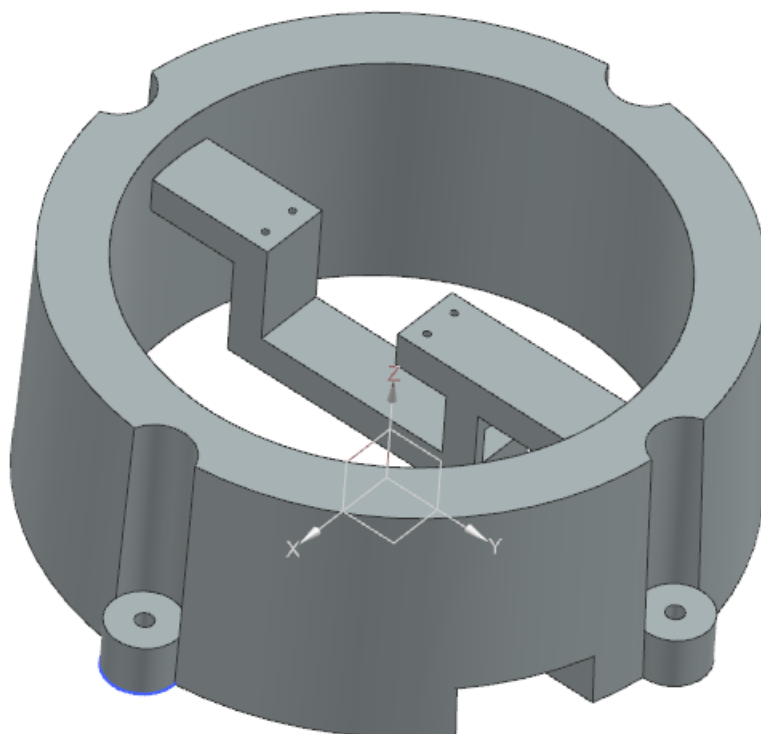
5.3.1 Podstawa

Podstawa stanowi główny element nośny całej konstrukcji manipulatora. Jej zadaniem jest zapewnienie stabilnego mocowania pierwszego serwomechanizmu odpowiedzialnego za obrót manipulatora wokół osi pionowej oraz przenoszenie obciążeń na podłoże.

Konstrukcja podstawy ma kształt cylindrycznej obudowy z wewnętrznymi wspornikami wyposażonymi w otwory montażowe, umożliwiające pewne zamocowanie serwomechanizmu. Rozwiązanie to zapewnia odpowiednie ustalenie jego położenia oraz ogranicza możliwość przemieszczania się podczas pracy.

Na zewnętrznej części podstawy znajdują się cztery punkty montażowe z otworami przelotowymi, umożliwiające trwałe przykręcenie manipulatora do stanowiska roboczego.

W konstrukcji przewidziano również odpowiednio rozmieszczone przestrzenie oraz wycięcia umożliwiające prowadzenie przewodów zasilających i sygnałowych. Dzięki temu okablowanie może zostać poprowadzone wewnątrz podstawy w uporządkowany sposób, co zmniejsza ryzyko jego uszkodzenia oraz zapobiega kolizji przewodów z ruchomymi elementami manipulatora.



Rysunek 7. Model podstawy.

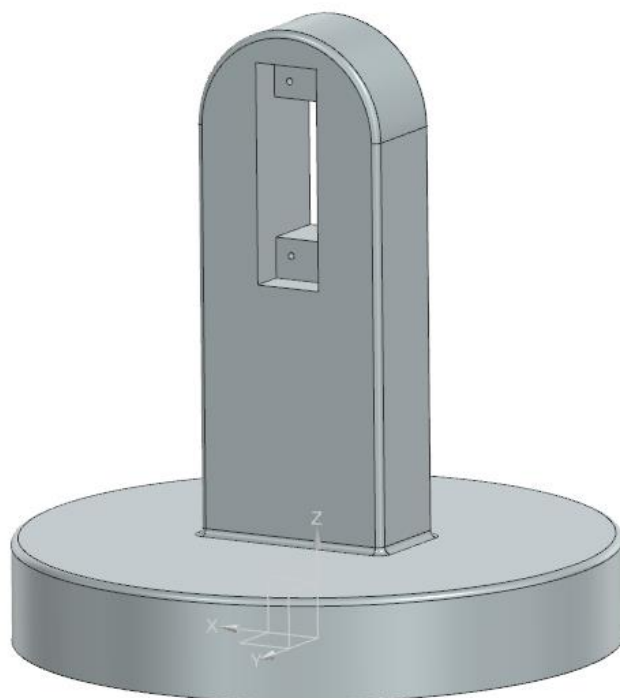
5.3.2 Pierwszy człon manipulatora

Pierwszy człon manipulatora stanowi element łączący podstawę z drugim członem ramienia. Jego zadaniem jest przenoszenie obciążeń powstających podczas pracy manipulatora oraz zapewnienie stabilnego mocowania drugiego serwomechanizmu odpowiedzialnego za ruch kolejnego członu.

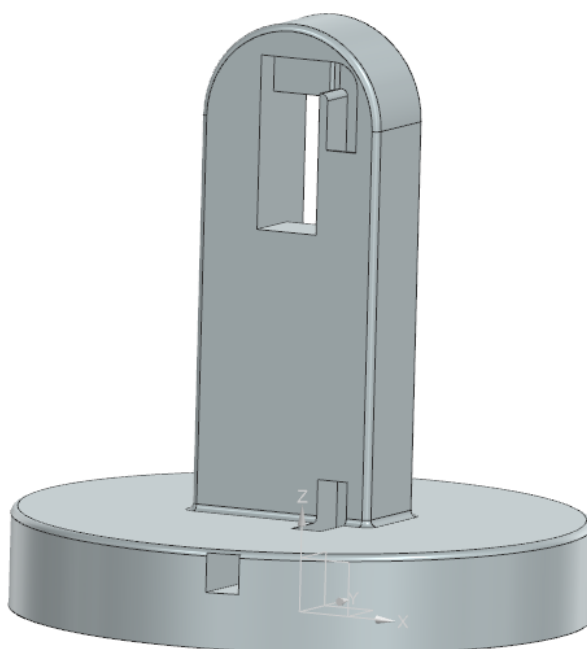
W górnej części elementu znajduje się otwór montażowy umożliwiający osadzenie serwomechanizmu oraz wykonanie połączenia obrotowego z następnym członem. Rozmieszczenie otworów montażowych zapewnia prawidłowe ustalenie położenia serwa oraz pewne zamocowanie całego zespołu.

W dolnej części elementu zaprojektowano powierzchnię montażową umożliwiającą trwałe połączenie z podstawą manipulatora. Konstrukcja została odpowiednio usztywniona, dzięki czemu zapewnia wysoką sztywność przy zachowaniu stosunkowo niewielkich wymiarów.

Dodatkowo w konstrukcji przewidziano przełotki oraz kanały umożliwiające prowadzenie przewodów. Takie rozwiązanie pozwala na estetyczne ukrycie okablowania, ogranicza ryzyko jego uszkodzenia oraz zapobiega kolizji przewodów z ruchomymi częściami ramienia podczas pracy.



Rysunek 8. Model pierwszego członu.



Rysunek 9. Model pierwszego członu.

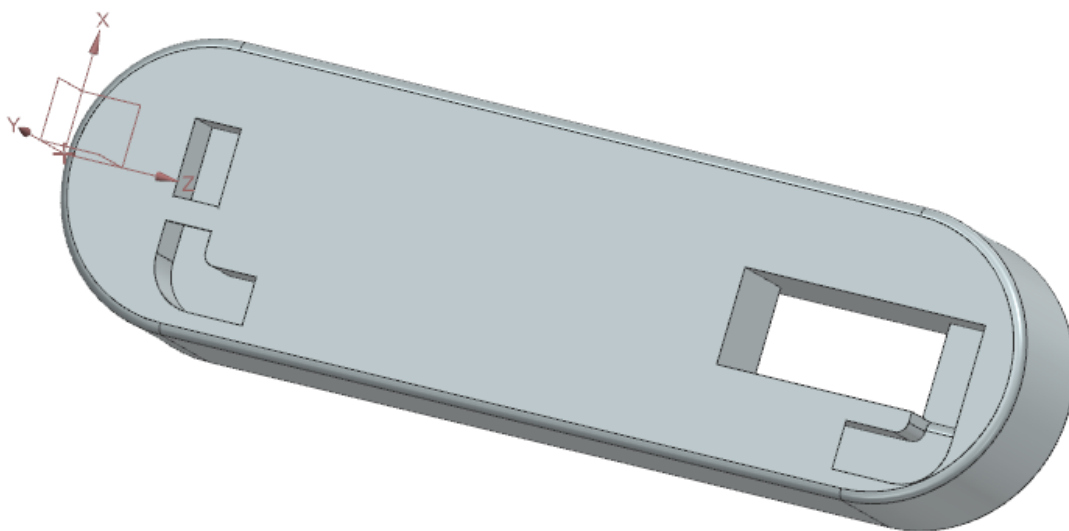
5.3.3 Drugi człon manipulatora

Drugi człon manipulatora stanowi element łączący pierwszy człon z końcową częścią ramienia. Jego zadaniem jest przenoszenie obciążeń pomiędzy kolejnymi osiami manipulatora oraz zapewnienie odpowiedniej sztywności konstrukcji podczas wykonywania ruchów.

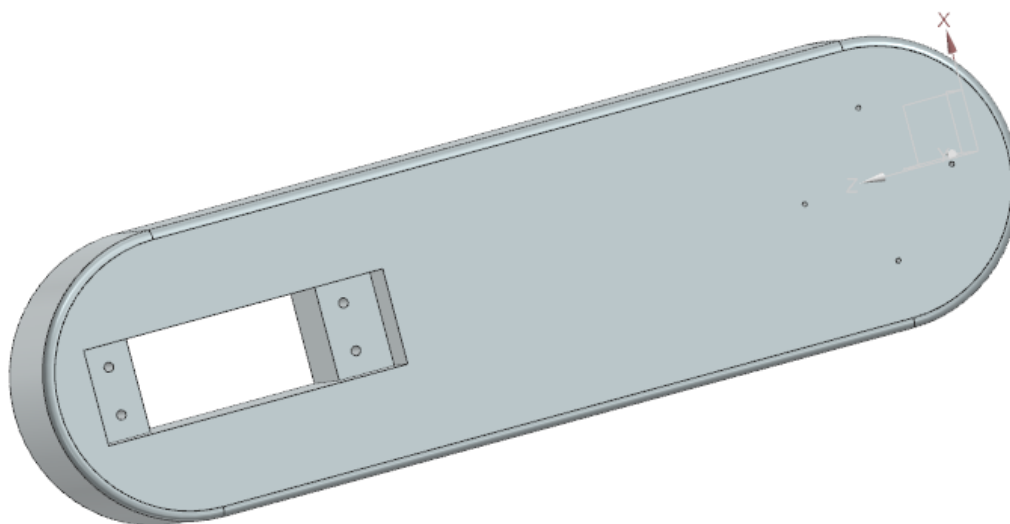
Na obu końcach elementu zaprojektowano miejsca montażowe umożliwiające połączenie z pozostałymi członami manipulatora. Otwory montażowe zostały rozmieszczone w sposób zapewniający prawidłowe pozycjonowanie elementów oraz pewne zamocowanie połączeń śrubowych.

W konstrukcji wykonano również odpowiednio rozmieszczone wycięcia technologiczne, które umożliwiają montaż elementów napędowych oraz ograniczają masę członu bez istotnego wpływu na jego wytrzymałość. Dzięki temu zmniejszono moment bezwładności ramienia oraz obciążenie serwomechanizmów.

Dodatkowo przewidziano przełotki umożliwiające prowadzenie przewodów pomiędzy kolejnymi członami manipulatora.



Rysunek 10. Model drugiego członu.



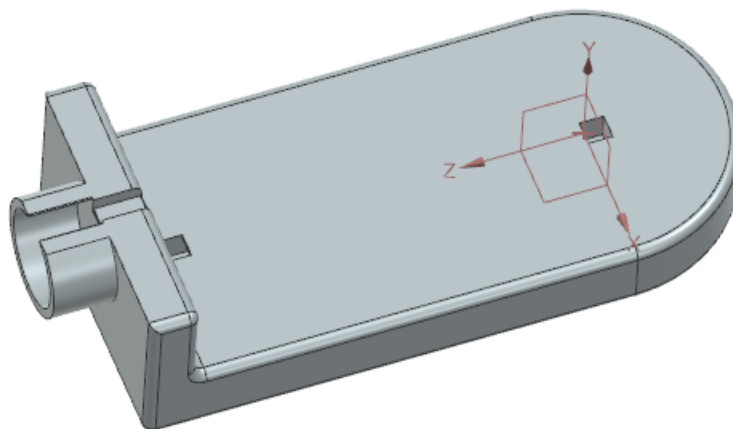
Rysunek 11. Model drugiego członu.

5.3.4 Trzeci człon manipulatora z miejscem na elektromagnes

Ostatni człon manipulatora stanowi element końcowy konstrukcji, którego zadaniem jest połączenie drugiego członu z uchwytem elektromagnesu. Odpowiada za przenoszenie obciążeń występujących podczas chwytania i transportu elementów oraz zapewnia odpowiednie pozycjonowanie efektora końcowego.

Na jednym z końców elementu znajduje się miejsce montażowe umożliwiające wykonanie połączenia obrotowego z poprzednim członem manipulatora. Drugi koniec został wyposażony w uchwyt z cylindrycznym gniazdem przeznaczonym do zamocowania elektromagnesu. Rozwiązanie to zapewnia stabilne osadzenie efektora oraz jego właściwe ustawienie względem osi ramienia.

Konstrukcja elementu została zaprojektowana z zachowaniem odpowiedniej sztywności przy jednoczesnym ograniczeniu masy własnej, co pozwala zmniejszyć obciążenie serwomechanizmu odpowiedzialnego za ruch końcowego członu. Dodatkowo przewidziano przelotkę umożliwiającą poprowadzenie przewodów zasilających elektromagnes. Dzięki temu okablowanie prowadzone jest w uporządkowany sposób, jest zabezpieczone przed uszkodzeniami mechanicznymi oraz nie ogranicza zakresu ruchu manipulatora.



Rysunek 12. Model trzeciego członu wraz z miejscem na elektromagnes.

5.4 Elementy złączne

Do montażu poszczególnych elementów manipulatora wykorzystano śruby dołączone przez producenta do zastosowanych serwomechanizmów. Śruby te posłużyły do mocowania serw w przygotowanych gniazdach oraz do połączenia orczyków z elementami konstrukcji ramienia. Rozmieszczenie otworów montażowych zostało dostosowane do wymiarów wykorzystanych elementów złącznych, co umożliwiło szybki montaż i ewentualny demontaż manipulatora w celu konserwacji lub wymiany podzespołów.

5.5 Proces montażu manipulatora

W niniejszym rozdziale przedstawiono kolejne etapy montażu manipulatora. Opisana kolejność prac pozwala na prawidłowe złożenie wszystkich elementów konstrukcji oraz może stanowić instrukcję montażu dla osób wykonujących manipulator na podstawie niniejszej dokumentacji.

1. Zamocować serwomechanizmy w miejscach montażowych każdego z członów.
2. Zamocować orczyki w miejscach montażowych każdego z członów.
3. Zamontować pierwsze ramię.
4. Zamontować drugie ramię.
5. Zamocować trzecie ramię.

6. Przykręcić elektromagnes w miejscu mocowania.
7. Poprowadzić przewody przez przygotowane otwory.
8. Sprawdzić swobodę ruchu wszystkich osi.

5.6 Gotowy zmontowany manipulator

Na rysunku przedstawiono kompletnie zmontowany manipulator wraz z serwomechanizmami, elektromagnesem oraz okablowaniem. Wykonana konstrukcja odpowiada opracowanemu modelowi 3D i potwierdza możliwość wykonania urządzenia na podstawie przygotowanej dokumentacji.

FOTA

Po wykonaniu i złożeniu manipulatora stwierdzono, że model 3D w dużym stopniu odpowiada rzeczywistej konstrukcji, a wszystkie elementy zostały poprawnie dopasowane pod względem funkcjonalnym. Podczas montażu konieczne było jednak wprowadzenie kilku drobnych modyfikacji wynikających z tolerancji wykonania elementów.

Największym problemem okazało się niewystarczające powiększenie gniazd przeznaczonych do montażu serwomechanizmów. W praktyce wymagały one niewielkiego zeszlifowania, aby możliwe było swobodne osadzenie serw. Dodatkowo, ze względu na różnicę wysokości pomiędzy pierwszym członem a orczykiem pierwszego serwomechanizmu, konieczne było wtopienie orczyka w pierwszy człon, co pozwoliło uzyskać prawidłowe położenie elementów i zapewnić poprawną pracę mechanizmu.

Na podstawie doświadczeń zdobytych podczas montażu można wskazać kilka usprawnień, które warto uwzględnić w kolejnej wersji projektu. Przede wszystkim należałoby nieznacznie zwiększyć wymiary otworów montażowych dla serwomechanizmów, aby wyeliminować konieczność ich ręcznej obróbki. Drugą proponowaną zmianą jest modyfikacja sposobu mocowania orczyków poprzez wykonanie w kolejnych członach dedykowanych gniazd odpowiadających ich kształtowi. Takie rozwiązanie ułatwi montaż, poprawi pozycjonowanie elementów oraz zwiększy sztywność połączeń.

6. Wnioski

W pierwotnej fazie projektowej do poruszania członami manipulatora wytypowano mniejsze serwomechanizmy. Jednakże podczas testów obciążeniowych prototypu doszło do awarii dwóch z nich. Analiza powypadkowa wykazała, że zjawisko to było spowodowane zbyt niskim znamionowym momentem obrotowym silników w stosunku do sił działających na ramiona w skrajnych pozycjach geometrycznych (maksymalne wyciągnięcie efektora końcowego wraz z masą własną elektromagnesu tworzyło zbyt długą dźwignię, przekraczającą moment trzymający silnika).

W celu eliminacji tego problemu podjęto decyzję o rekonfiguracji układu wykonawczego. Słabsze jednostki zastąpiono mocniejszymi serwomechanizmami o wyższym standardzie – modelem MG995.

Wykorzystanie bazy danych do sterowania oraz stworzenie strony internetowej odpowiedzialnej za sterowanie pozwala na dużą swobodę oraz pozwala na dużą swobodę oraz asynchroniczną wymianę danych między użytkownikiem a robotem. Mikrokontroler nie musi utrzymywać bezpośredniego i stałego połączenia z przeglądarką operatora lecz odpytuje serwer w chmurze o aktualny stan zapamiętany w strukturze danych (np. słowniku RAM lub bazie danych), co minimalizuje obciążenie układu wbudowanego.